



Parallel Connected Components

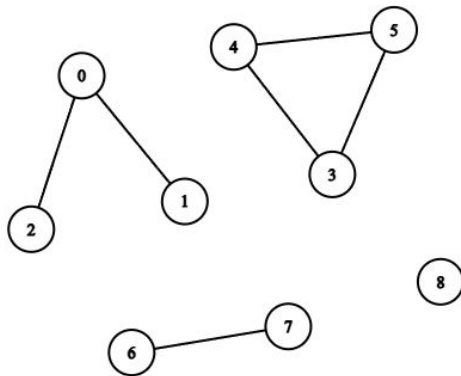
Salvatore Gilles Cassarà, Alberto Cimmino

Problem Introduction

In graph theory, a connected component of an undirected graph G is a maximal set of connected nodes of G .

Many real-time applications:

- Image segmentation
- Clustering of 3D points
- Tracking communities in social media streams

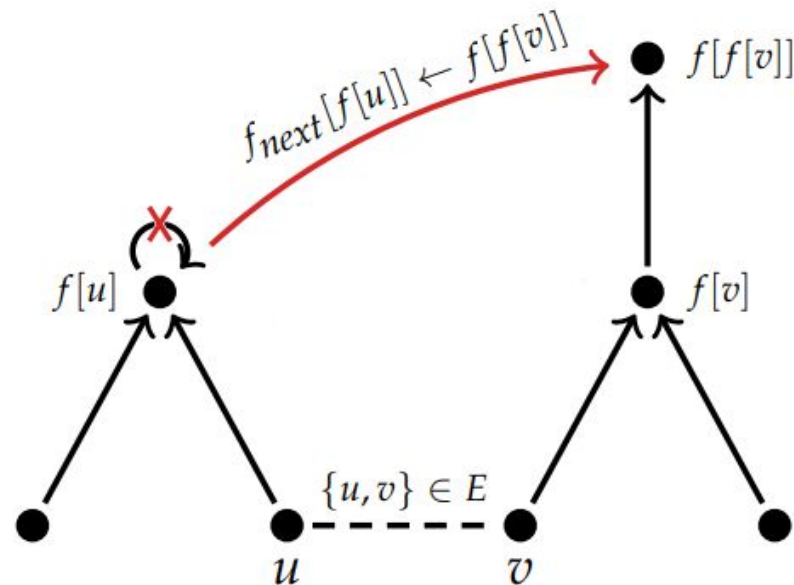


Sequential Solution

Based on the **Shiloach-Vishkin (SV)** algorithm

Split in 3 phases:

- Stochastic Hooking
 - Find minimum grandparent among the neighbours' grandparents
- Shortcutting
 - Flatten the tree by setting parent of node to minimum between parent and grandparent
- Convergence Check



Modified image from: [FastSV paper](#)



Current State of the Art

Adaptive Strategy

- If graph is small, BFS is faster than SV. And vice versa.
- Heuristics are used to determine which to use

FastSV

Improvements on SV:

- New hooking strategies
- Linear algebra

Map problem to *Contour* algorithm

- Compute the “border” of a connected component
- Faster than FastSV



Parallel Implementation

Based on **FastSV algorithm**, using both MPI and OpenMP.

Flow of the program:

- Split data among the processes
- Loop until convergence:
 - Each process calculate a step of the calculation
 - Each process shares their data and receives new parent vector



Parallel Implementation | Split Criteria

Problem: Achieve even workload and rows of the matrix can't be divided

Solution: Find ideal number of edges per MPI process and assign full rows while staying as close as possible to it

Calculated by process 0

- *MPI_Bcast* to share information
- *MPI_Scatter* and *MPI_Scatterv* to share data



Parallel Implementation | Computation

Improvements on SV algorithm:

- **Neighbor-wise Propagation**
 - Calculate the minimum current grandparent label among itself and its neighbors
- **Aggressive Hooking**
 - Short the path to root, by reusing the stochastic hooking result
- **Pointer-Jumping**
 - To reduce the tree height, the grandparent array is updated after each iteration



Parallel Implementation | Parallelization

MPI

At each step, the *full* parent vector has to be communicated between process

Improvements:

- CSR-like storage format
- Communicate every 4 steps (Heuristic)

OpenMP

Algorithm is dominated by **for-loops** whose iterations are mostly independent.

Parallel Implementation | Computation Pseudocode

1. Compute next step

```
12: OpenMP parallel region
13: while global_sum  $\neq$  0 do
14:   for step = 1 to BATCH_SIZE do
15:     findMinGrandparentOfNeighbours(graph, gp, mngp)
16:     OpenMP parallel for
17:       f[f[u]]  $\leftarrow$  atomic(min(f[f[u]], mngp[u]))
18:     OpenMP parallel for
19:       f[u]  $\leftarrow$  min(f[u], mngp[u], gp[u])
```

$\triangleright$ 1. Neighbor-wise propagation: $mngp \leftarrow graph \otimes gp$

$\triangleright$ 2. Stochastic hooking

$\triangleright$ 3. Aggressive hooking + shortcutting

2. Share parent array

```
20: OpenMP master
21: if step = BATCH_SIZE then
22:   MPI_Allgatherv
23:   f  $\leftarrow$  buffer
24: end if
25: OpenMP barrier
```

$\triangleright$ 4. Global parent synchronization

3. Grandparent and convergence

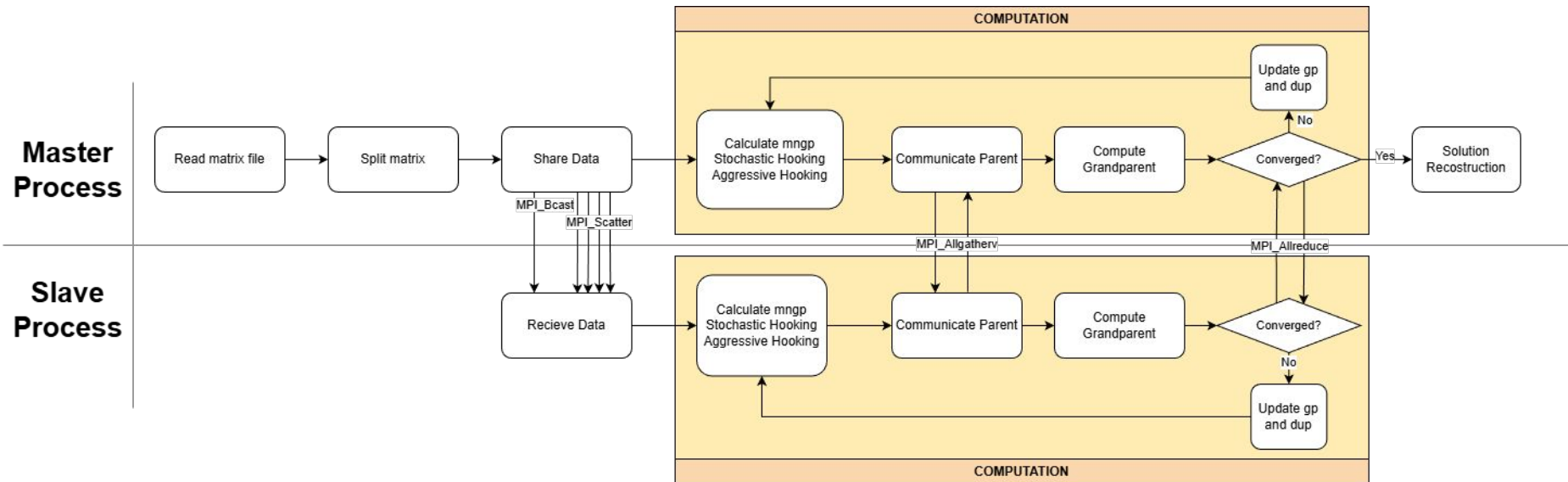
```
26: OpenMP parallel for
27:   gp[u]  $\leftarrow$  min(f[f[u]], buffer[f[u]])
28: OpenMP master
29: if step = BATCH_SIZE then
30:   local_diff  $\leftarrow$  (gp  $\neq$  dup)
31:   MPI_Allreduce(local_diff, global_sum, +)
32: else
33:   global_sum  $\leftarrow$  1
34: end if
35: OpenMP parallel for
36:   dup[u]  $\leftarrow$  gp[u]
37: end for
38: end while
39: Free auxiliary vectors
40: end procedure
```

$\triangleright$ 5. Pointer-Jumping

$\triangleright$ 6. Global convergence check

$\triangleright$ 7. Update state

Parallel Implementation | Flow





Data Dependency

Variable	Type of Access	Visibility
<i>f[]</i>	Read / Write	Shared
<i>gp[]</i>	Read / Write	Shared
<i>dup[]</i>	Read / Write	Shared
<i>mngp[]</i>	Read / Write	Shared
<i>buffer[]</i>	Read / Write	Shared
<i>global_sum</i>	Read / Write	Shared
<i>local_diff</i>	Read / Write	Private (only master)
<i>reduce_result</i>	Read / Write	Private (only master)



Data Dependency

Variable	Type of Access	Visibility
<i>f[]</i>	Read / Write	Shared
<i>gp[]</i>	Read / Write	Shared
<i>dup[]</i>	Read / Write	Shared
<i>mngp[]</i>	Read / Write	Shared
<i>buffer[]</i>	Read / Write	Shared
<i>global_sum</i>	Read / Write	Shared
<i>local_diff</i>	Read / Write	Private (only master)
<i>reduce_result</i>	Read / Write	Private (only master)



Data Dependency

Variable	Type of Access	Visibility
<i>f[]</i>	Read / Write	Shared
<i>gp[]</i>	Read / Write	Shared
<i>dup[]</i>	Read / Write	Shared
<i>mngp[]</i>	Read / Write	Shared
<i>buffer[]</i>	Read / Write	Shared
<i>global_sum</i>	Read / Write	Shared
<i>local_diff</i>	Read / Write	Private (only master)
<i>reduce_result</i>	Read / Write	Private (only master)



Data Dependency

Variable	Type of Access	Visibility
<i>f[]</i>	Read / Write	Shared
<i>gp[]</i>	Read / Write	Shared
<i>dup[]</i>	Read / Write	Shared
<i>mngp[]</i>	Read / Write	Shared
<i>buffer[]</i>	Read / Write	Shared
<i>global_sum</i>	Read / Write	Shared
<i>local_diff</i>	Read / Write	Private (only master)
<i>reduce_result</i>	Read / Write	Private (only master)



Performance Analysis | Setup

- HPC2 Cluster
- Placing strategy: **Exclusive pack**
- Each execution:
 - 10 runs
 - Tested configurations:
 - Hybrid (MPI+OpenMP)
 - MPI
 - OpenMP
 - With 2, 4, 8, 16, 32, 64 workers



Performance Analysis | Setup

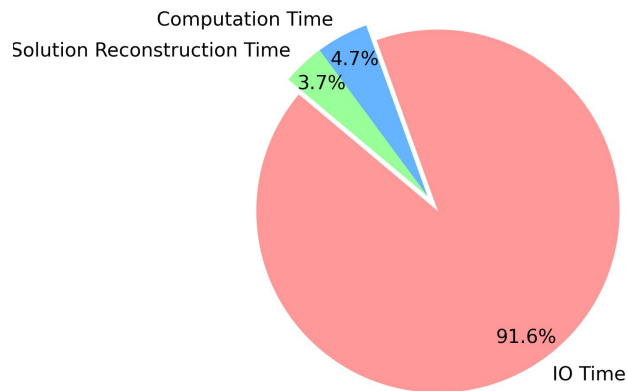
- HPC2 Cluster
- Placing strategy: **Exclusive pack**
- Each execution:
 - 10 runs
 - Tested configurations:
 - Hybrid (MPI+OpenMP)
 - MPI
 - OpenMP
 - With 2, 4, 8, 16, 32, 64 workers

Name	# Nodes	# Edges	# CCs
<i>double</i>	100,000,000	396,975,220	629,462
<i>standard</i>	50,000,000	178,548,916	4,750,265
<i>halved</i>	25,000,000	92,019,612	1,750,265

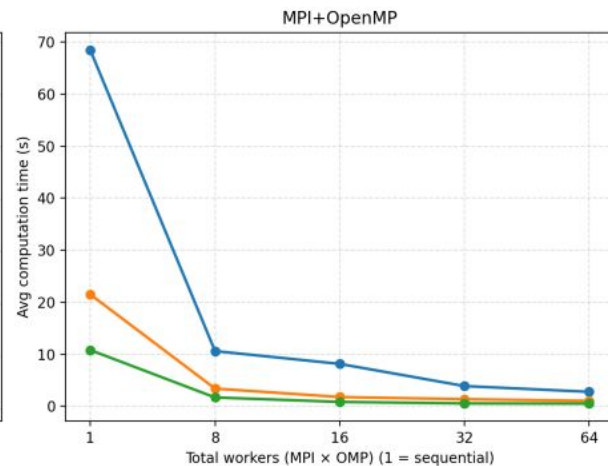
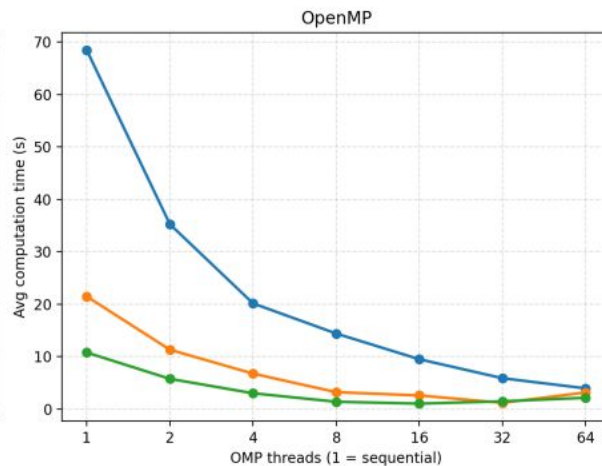
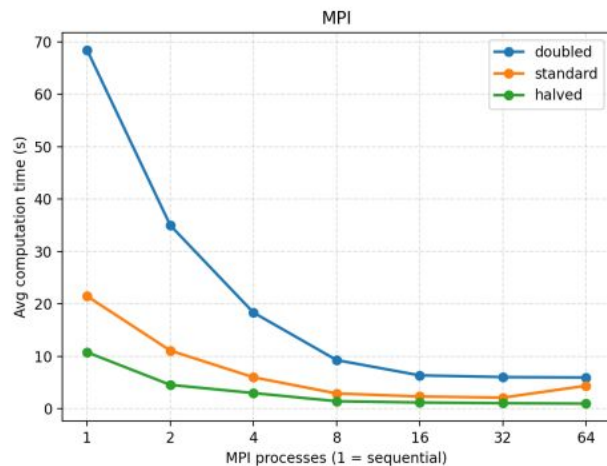
Performance Analysis | Setup

- HPC2 Cluster
- Placing strategy: **Exclusive pack**
- Each execution:
 - 10 runs
 - Tested configurations:
 - Hybrid (MPI+OpenMP)
 - MPI
 - OpenMP
 - With 2, 4, 8, 16, 32, 64 workers

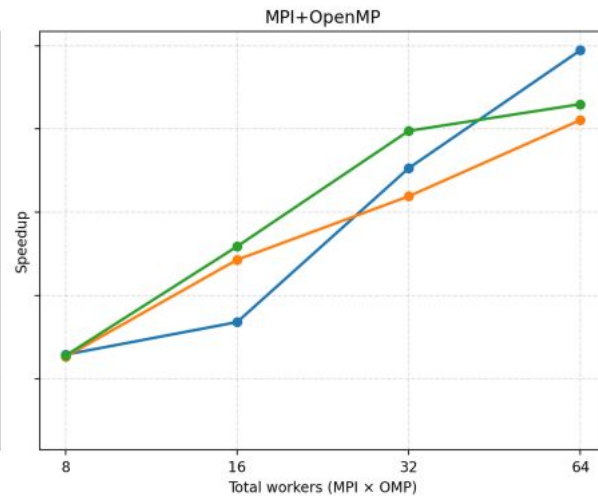
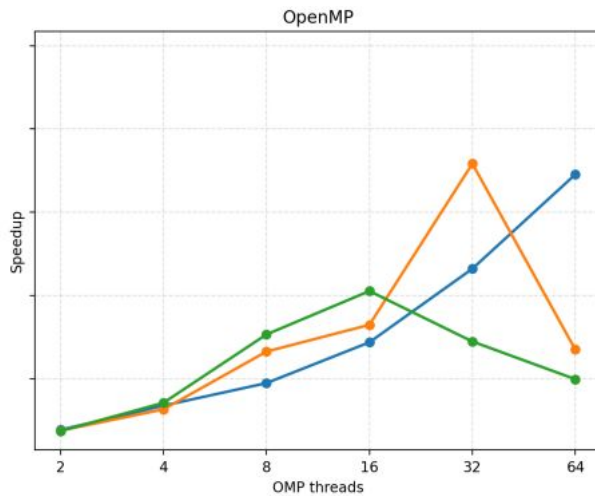
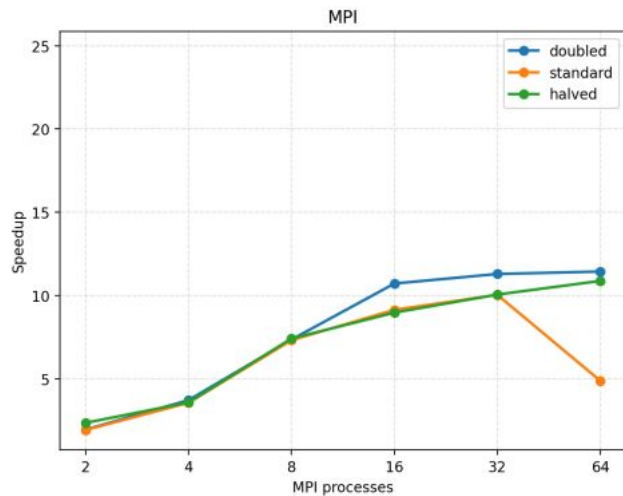
Name	# Nodes	# Edges	# CCs
<i>double</i>	100,000,000	396,975,220	629,462
<i>standard</i>	50,000,000	178,548,916	4,750,265
<i>halved</i>	25,000,000	92,019,612	1,750,265



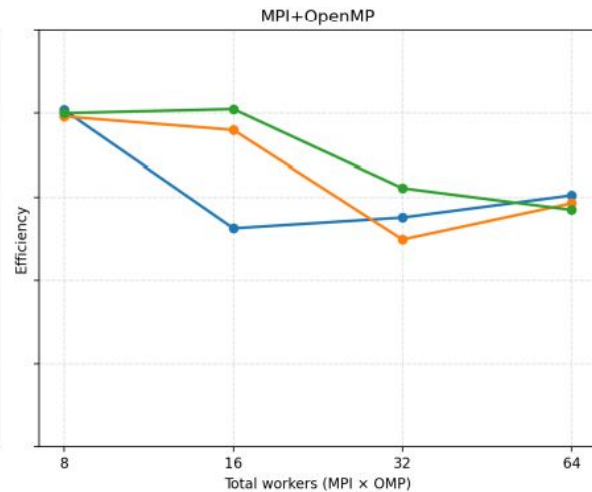
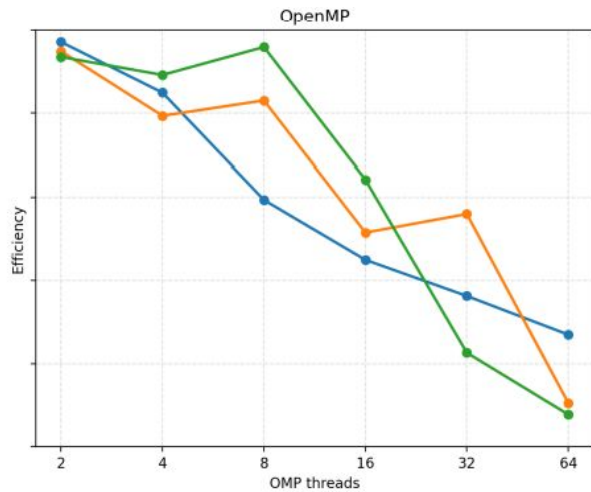
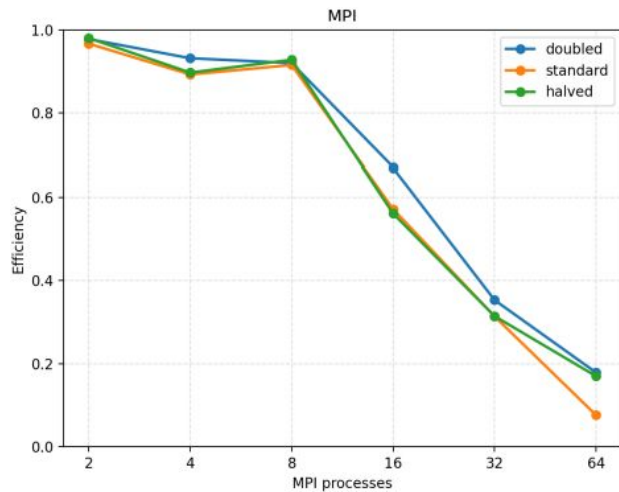
Performance Analysis | Computation Time



Performance Analysis | Speedup



Performance Analysis | Efficiency



Parallel Implementation | Computation Pseudocode

1. Compute next step

```
12: OpenMP parallel region
13: while global_sum  $\neq$  0 do
14:   for step = 1 to BATCH_SIZE do
15:     findMinGrandparentOfNeighbours(graph, gp, mngp)
16:     OpenMP parallel for
17:       f[f[u]]  $\leftarrow$  atomic(min(f[f[u]], mngp[u]))
18:     OpenMP parallel for
19:       f[u]  $\leftarrow$  min(f[u], mngp[u], gp[u])
```

$\triangleright$ 1. Neighbor-wise propagation: $mngp \leftarrow graph \otimes gp$

$\triangleright$ 2. Stochastic hooking

$\triangleright$ 3. Aggressive hooking + shortcutting

2. Share parent array

```
20: OpenMP master
21: if step = BATCH_SIZE then
22:   MPI_Allgather
23:   f  $\leftarrow$  buffer
24: end if
25: OpenMP barrier
```

$\triangleright$ 4. Global parent synchronization

3. Grandparent and convergence

```
26: OpenMP parallel for
27:   gp[u]  $\leftarrow$  min(f[f[u]], buffer[f[u]])
28: OpenMP master
29: if step = BATCH_SIZE then
30:   local_diff  $\leftarrow$  (gp  $\neq$  dup)
31:   MPI_Allreduce(local_diff, global_sum, +)
32: else
33:   global_sum  $\leftarrow$  1
34: end if
35: OpenMP parallel for
36:   dup[u]  $\leftarrow$  gp[u]
37: end for
38: end while
39: Free auxiliary vectors
40: end procedure
```

$\triangleright$ 5. Pointer-Jumping

$\triangleright$ 6. Global convergence check

$\triangleright$ 7. Update state



Performance Analysis | Efficiency

	Halved	Standard	Doubled
8	0.799662	0.791772	0.807045
16	0.809456	0.759525	0.525578
32	0.621516	0.498760	0.551511
64	0.569914	0.585663	0.604609



Scalability Analysis

Not **Weakly** scalable

	Halved	Standard	Doubled
8	0,799662	0,791772	0,807045
16	0,809456	0,759525	0,525578
32	0,621516	0,498760	0,551511
64	0,569914	0,585663	0,604609



Scalability Analysis

Almost **Strongly** scalable

	Halved	Standard	Doubled
8	0,799662	0,791772	0,807045
16	0,809456	0,759525	0,525578
32	0,621516	0,498760	0,551511
64	0,569914	0,585663	0,604609



Conclusions

- Average Efficiency: 66% ($\leq 70\%$)
- Implemented without using libraries
- Difficult to implement, as it is a **global problem**
 - Requires Strong Synchronization
- Future improvements
 - Explore different data communication patterns
 - Data compression



Parallel Connected Components

Salvatore Gilles Cassarà, Alberto Cimmino

DENSE MATRIX

	0	1	2	3
0	1.0		2.0	
1		3.0		
2				
3	4.0	5.0		
4		6.0	7.0	8.0

COMPRESSED SPARSE ROW - CSR (ZERO-BASE INDEX)

ROW
OFFSETS

0	1	2	3	4	5
0	2	3	3	5	8

COLUMN
INDICES

0	1	2	3	4	5	6	7
0	2	1	0	1	1	2	3

VALUES

0	1	2	3	4	5	6	7
1.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0